

PRODUCT REQUIREMENTS DOCUMENT

Simplitest

A Test Management Platform for SimpliEd QA

Status: Final v1.0

Prepared by: Ansa Fatima, QA Engineering

Date: September 2, 2026

Audience: Engineering & QA management

1. Executive summary

Simplitest is a purpose-built test management platform for SimpliEd's QA organization. It replaces spreadsheets and disconnected trackers with a single workspace for organizing test cases, running both structured regression cycles and lightweight ad hoc checks, and reporting on release readiness and long-term module stability across every SimpliEd product surface — Admin Portal, Teacher/Parent/Student apps, QR Attendance, and any portal added later.

This document describes the problem it solves, who it's for, and what it does today, and lays out what's intentionally out of scope for now.

2. Problem statement

- Test cases and results were scattered across spreadsheets and documents, with no single source of truth for what had been tested or what passed.
 - There was no consistent way to see a module's pass/fail history over time, so instability crept in quietly instead of showing up as an early warning before release.
 - Managers had no self-serve visibility into QA coverage, tester workload, or release health — status required manually pinging QA.
 - Ad hoc, same-day testing was too heavy to log formally, so it often went untracked entirely.
 - Onboarding a new tester meant explaining ad hoc conventions instead of pointing them at a shared structure.
-

3. Goals

- **Single source of truth:** every portal/app has one structured home for its test cases (Portal → Module → Suite → Test Case).
 - **Two speeds of testing:** support full case-by-case regression cycles and 30-second quick logs, so lightweight testing gets recorded instead of skipped.
 - **Early warning on instability:** surface which modules are trending toward instability before they become a release incident.
 - **Self-serve reporting:** give managers and leads execution, release, and stability reports without asking QA for manual updates.
 - **Low-friction team admin:** invites, roles, and account recovery that don't depend on email infrastructure.
-

4. Non-goals (out of scope for this version)

- Automated test execution or CI test-runner integration — today's runs are logged manually, whether or not the testing itself was automated.
 - File/screenshot/video evidence attachments on test runs — shown as “Coming soon” in the UI, not yet built.
 - Replacing a defect tracker — issue counts and severities are tracked, but there's no ticket workflow; a ticket-link field points out to an external tracker.
 - Real-time collaboration beyond per-run notes (no comment threads or @mentions).
 - A native mobile app — the web app is responsive, not a packaged app.
 - Dedicated tester-performance and per-module coverage reports — descoped to keep the report set focused on release-readiness and stability, the two questions managers actually ask.
-

5. Target users

- **QA Tester** — logs test results (case-based or quick log), files issues, and tracks their own execution history.
 - **QA Manager / Lead** — plans test cycles, defines scope, reviews reports, and signs off releases based on stability data.
 - **Developer** — checks failures relevant to their module and the stability trend of an area before making changes.
 - **Super Admin** — manages workspace membership, roles, and account recovery for the whole team.
 - **Viewer / Stakeholder** — read-only access to reports and dashboards to check release health without a QA background.
-

6. Product structure

Test content is organized in a fixed hierarchy so every portal has a predictable home for its cases:

Workspace → Portal (app/product) → Module (feature area) → Suite (nested folder, any depth) → Test Case

A test cycle then runs against that structure in one of two modes:

- **Case-based:** a scoped regression run — all cases, a whole portal, a module, a suite, or a hand-picked custom list — executed case by case as Passed / Failed / Blocked / Skipped.
 - **Manual quick log:** a single aggregate pass/fail verdict plus issue-severity counts logged against a module or feature in seconds, for testing that doesn't warrant a full regression pass.
-

7. Core features & requirements

7.1 Test case management

- Structured Portal / Module / Suite hierarchy with unlimited nested folders.
- Full case detail: preconditions, steps, expected result, priority, severity, type, status, and an optional owner.
- Search and filter by priority and type — kept deliberately minimal so every filter maps to something visible in the table.
- Bulk actions: bulk edit fields, bulk move between locations, duplicate, and delete.
- CSV import for bringing in existing suites.

7.2 Test execution

- Case-based cycles with a scope picker (all / portal / module / suite / custom selection).
- Manual quick logs for fast, aggregate-only testing.
- Live progress per cycle — executed vs. remaining, pass/fail/blocked breakdown.
- Sorting and filtering of test runs by date (auto-sorted newest first), portal, module, and status.

7.3 Reporting & analytics

- Dashboard — total cases, pass rate, open failures, active runs, an execution trend chart, and a Recent activity feed of the latest test cycles and quick logs.
- Execution report — test run results over a chosen window (7/30/90 days, 12 months, or all time), with a daily or weekly trend chart that always reconciles exactly with the KPI totals above it, however wide the window.
- Release report — sprint-level pass/fail summary per cycle to support go/no-go decisions.
- **Stability report** — blends case-based runs and quick logs into a rolling pass rate per module/feature; classifies each as Stable, At Risk, Unstable, or No data; shows a trend (improving / declining / flat). Clicking any module or feature opens a drill-down panel with its pass-rate trend chart and its failed test cycles/quick logs — grouped by run, not by individual test case — each linking straight to that run.
- CSV/PDF export and a shareable link on every report.

7.4 Team & access management

- Role-based access across five roles — Super Admin, QA Manager, Tester, Developer, Viewer — at both workspace and per-project level.
- Invite-link onboarding — no email server required.
- Admin-mediated password reset — a Super Admin generates a one-time reset link for a locked-out teammate, without ever seeing or setting their password directly.

8. Success metrics

- Share of releases with a completed case-based regression cycle before go-live.
- Modules that reach “At Risk” before they reach “Unstable” — i.e., the trend gives a real early warning instead of a same-day surprise.

- Time from “testing needed” to “logged” for ad hoc work, as a proxy for quick-log adoption.
- Manager self-serve report views vs. manual status-update requests sent to QA.

9. Technical overview

Built on Next.js 14 (App Router) with TypeScript and Tailwind CSS, backed by PostgreSQL via Prisma. One codebase serves both the web application and its API. Deployment is continuous: pushes to the main branch run through GitHub Actions and deploy automatically via CapRover.

10. Roadmap / future considerations

- Evidence attachments (screenshots, video) on test runs.
 - CI / automated test-runner integration.
 - Native mobile app or installable PWA.
 - Two-way defect-tracker integration.
-

Appendix A: Feature status summary

Area	Status
Test case management	Live — hierarchy, bulk edit/move, search & filter, CSV import
Case-based test cycles	Live — scoped execution (all / portal / module / suite / custom pick)
Manual quick logs	Live — lightweight aggregate logging, feeds the Stability report
Dashboard	Live — KPIs, execution trend chart, Recent activity feed
Execution report	Live — 7/30/90-day, 12-month, and all-time windows
Release report	Live — sprint-level pass/fail summary
Stability report	Live — trend + drill-down panel to failed runs by module/feature
Team & role management	Live — invite links, 5-tier roles, admin-mediated password reset
Evidence attachments (screenshots/video)	Planned — placeholder in UI today
CI / automated test-runner integration	Not started
Two-way defect-tracker sync	Not started